

EliteRestaurant Technical Appendix

This appendix complements the main documentation with deeper implementation detail. It is intended as a maintenance, onboarding, and audit reference for developers or operators who need more than a conceptual overview.

Scope: repository inventory, startup flow, project dependencies, endpoint reference, entity reference, settings schema, workflow details, tooling notes, and technical observations.

A. Repository Inventory

Area	Representative files or folders	What lives there
Desktop shell	EliteRestaurantPro/App.xaml.cs , MainWindow.xaml	Application startup, exception logging, import mode, theme initialization, root navigation host.
Desktop screens	EliteRestaurantPro/Views	WPF pages for employees, tables, reservations, orders, money, payroll, reports, login, settings, and kitchen/server work.
Desktop logic	EliteRestaurantPro/ViewModels	MVVM behavior, commands, navigation, business logic integration, dashboard and operational workflows.
Shared persistence	EliteRestaurantPro/Data/AppDbContext.cs	Entity model, indexes, schema patching, bootstrap helpers, active-order reconciliation, legacy import logic.
Shared utilities	EliteRestaurantPro/Utils	Settings, currency, workflow constants, payroll math, alerts, shared drafts, theming, export helpers.
API surface	EliteRestaurant.Api/Controllers	Authentication, server portal data and actions, tables, arrived reservations, and health checks.
API auth helpers	EliteRestaurant.Api/Security	Bearer parsing, in-memory token sessions, portal role candidate selection.
Web portal	EliteRestaurant.Api/wwwroot/index.html	Single-file browser UI with inline CSS and JavaScript.
Operational seeding	SeedRunner/Program.cs	Database reset and data generation tool.

Area	Representative files or folders	What lives there
Operational utility	Tools/ClearActiveOrders/Program.cs	Legacy-feeling active order cleanup entry point.

B. Project Dependencies and Coupling

Project	Direct dependencies	Architectural implication
EliteRestaurantPro	ClosedXML, EF Core SQLite, EF Core PostgreSQL, QuestPDF	Desktop app also acts as domain and persistence core; package list reveals reporting/export and dual-database history.
EliteRestaurant.Api	OpenAPI, Swashbuckle, project reference to EliteRestaurantPro	API is a thin host around the shared desktop/business project rather than an isolated service layer.
SeedRunner	Project reference to EliteRestaurantPro	Seeding logic directly manipulates the same live schema and entities as the application.
ClearActiveOrders	Project reference to EliteRestaurantPro	Utility relies on shared database code but appears partially out of sync with current implementation.

C. Startup Sequence Detail

C1. Desktop startup

Order	Action	Why it matters
1	Enable legacy Npgsql timestamp behavior.	Compatibility layer during date/time migration from older storage assumptions.
2	Register unhandled exception hooks.	Writes crash information to local logs and keeps operators informed.
3	Configure QuestPDF community license.	Enables PDF/report functionality.
4	Check for <code>--import-sqlite-now</code> .	Provides migration path from local legacy DB to PostgreSQL.
5	Ensure PostgreSQL connection string exists or prompt user.	Application is PostgreSQL-required in active mode.
6	Attempt database initialization in a retry loop.	Allows operator to correct a bad connection string at startup.
7	Apply theme palette and continue to UI shell.	Final presentation setup before view rendering.

C2. API startup

Order	Action	Effect
1	Enable legacy timestamp behavior.	Keeps DB date compatibility aligned with desktop startup.
2	Call <code>AppDbContext.Initialize()</code> .	API startup can create or patch schema before serving requests.
3	Add controllers, Swagger, auth service, CORS.	Configures REST surface and API discoverability.
4	Enable Swagger, CORS, default files, static files, controllers.	Serves both API and portal from the same host.

D. Entity Reference

D1. Staff and identity

Entity	Important fields	Behavioral notes
Employee	UniqueId , SignInId , Role , PinCode , ProfileImagePath , shift columns, rates	Used for both admin-side HR and tablet sign-in. Includes several non-mapped convenience properties for UI display and schedule summary.
EmployeeAttendance	Employee id, work date, clock-in/out, status, absence flags	Feeds payroll calculations and attendance review flows.
AttendanceDayValidation	Work date, validation timestamp	Supports attendance day confirmation logic.

D2. Sales and floor

Entity	Important fields	Behavioral notes
Table	Number, name, capacity, status, assigned server	Table status is periodically reconciled from active orders, except for maintenance tables.
OrderRecord	Status, server, notes, discount fields, payment fields, source, reservation linkage, created/completed timestamps	Acts as open check, kitchen workflow ticket, payment source, and eventual financial posting trigger.
OrderItem	Product, quantity, prep employee metadata	Each line can carry who should prepare it, usually chef for food and barman for drinks.

D3. Menu and inventory

Entity	Important fields	Behavioral notes
Product	Name, category, subcategory, price	Displayed both in desktop menu management and browser portal product listings.
InventoryItem	Name, unit, stock quantity, expiration date, notes	Supports stock management and ingredient mapping.
ProductIngredient	Product id, inventory item id, quantity	Bridge table between menu and inventory depletion logic.

D4. Finance and payroll

Entity	Important fields	Behavioral notes
MoneyTransaction	Amount, USD amount, FC amount, date, type, category, justification	Central finance ledger for revenue and expenses.
SalaryAdvance	Employee, amount, payroll month targeting, notes	Advances can be applied against future payroll runs.
PayrollPaymentRecord	Employee, year, month, salary, deductions, bonus, advances, net pay	Snapshot of a payroll calculation once paid.

D5. Reservation and customer relations

Entity	Important fields	Behavioral notes
CustomerProfile	Name, phone, contact preference, visit counters, notes	Supports reservation context and repeat guest tracking.
ReservationBooking	Guest, reserved time, channel, status, deposit, table, creator	Contains several computed convenience properties such as urgency, health state, and display name.
WaitlistEntry	Guest, phone, party size, quoted wait, status	Represents non-reservation queueing demand.

D6. Shared UI-state persistence

Entity	Important fields	Behavioral notes
SharedOrderDraft	Employee id, portal, label, payload JSON, timestamps	Stores complete server draft snapshots without normalizing cart or screen state.

E. Indexes and Relationship Notes

- Unique indexes exist for employee unique ids, product unique ids, table unique ids, and order unique ids.
- `Employee.SignInId` is unique when populated and non-empty.
- `Table.TableNumber` is unique.
- `ReservationBooking.ReservedFor` and status are indexed for time and state filtering.
- `SharedOrderDraft` is indexed by employee, portal, and update time for recent draft retrieval.
- Attendance uniqueness is enforced on employee plus work date.
- Payroll uniqueness is enforced on employee plus year plus month.

F. API Endpoint Reference

F1. Authentication

Method and route	Input	Success behavior	Failure behavior
POST <code>/api/auth/login</code>	<code>staffId</code> , <code>pin</code> , optional <code>portal</code>	Returns access token, expiration, employee identity, role, and sign-in id.	Unauthorized if id, pin, and portal role do not resolve to a valid active employee.

F2. Server portal configuration and media

Method and route	Purpose	Notes
GET <code>/api/server/config</code>	Return restaurant name, logo URL, employee photo URL, currency mode, exchange rate, tax and service settings.	Requires a valid server session.
GET <code>/api/server/assets/restaurant- logo?token=...</code>	Serve the restaurant logo file from local path stored in settings.	Uses token in query string rather than bearer header.
GET <code>/api/server/assets/me-photo? token=...</code>	Serve the logged-in employee photo file.	Reads file path from employee record in database.

F3. Server operational data

Method and route	Purpose	Key validation
GET <code>/api/server/products</code>	Return menu items ordered by category, subcategory, and name.	Server session required.
GET <code>/api/tables/my</code>	Return non-maintenance tables, filtered to assigned tables for server role.	Any valid session can call; server session narrows results.
GET <code>/api/server/open-check? tableId=...</code>	Determine whether the table has an open active order.	Table must exist and be assigned to current server.

F4. Shared drafts

Method and route	Purpose	Behavioral notes
GET <code>/api/server/drafts</code>	Return up to 50 recent drafts for the server employee.	Ordered by latest update first.
POST <code>/api/server/drafts</code>	Save a new JSON snapshot draft.	Request body must contain valid JSON in <code>SnapshotJson</code> .

Method and route	Purpose	Behavioral notes
DELETE <code>/api/server/drafts/{draftId}</code>	Delete a draft owned by the current employee.	Draft id must belong to the logged-in employee and server portal.

F5. Orders and ready queue

Method and route	Purpose	Important validation and side effects
POST <code>/api/server/orders</code>	Create a new order or append to an open check.	Checks table assignment, maintenance status, valid products, discount rules, delivery reference, and recalculates table occupancy and payment totals.
GET <code>/api/server/ready-orders</code>	Return ready orders for the current server.	Includes line summary and totals in both USD and FC.
POST <code>/api/server/orders/{orderId}/serve</code>	Mark a ready order as served.	Only the owning server can call it, and only from the <code>Ready</code> state.

F6. Reservations and health

Method and route	Purpose	Notes
GET <code>/api/reservations/arrived</code>	List recently arrived reservations.	Returns up to 100 rows with table label information.
GET <code>/api/health</code>	Process heartbeat endpoint.	No DB check beyond service liveness.
GET <code>/api/health/db</code>	Quick DB connectivity check with employee and table counts.	Returns HTTP 500 with exception message on failure.

G. Browser Portal Internal Logic

The browser portal is intentionally minimal and stores runtime state in JavaScript variables instead of browser persistence. The token is kept in a variable rather than `localStorage` or `sessionStorage`. As a result, a refresh or tab close will remove the active session from the UI side even if the API token still exists server-side.

Concern	Observed implementation
State storage	In-memory variables: token, current user, tables, products, ready orders, drafts, config, and cart.
HTTP wrapper	Single <code>api()</code> helper that adds bearer header when needed and parses JSON opportunistically.
Filtering	Client-side search plus category and subcategory filtering over preloaded product list.

Concern	Observed implementation
Totals	Client-side subtotal, discount, tax, service, and grand total calculation for immediate feedback.
Draft snapshots	Serializes current form state and cart to JSON, then restores from snapshot on load.
Order submission	Builds API payload directly from DOM state and cart contents.

H. Configuration Reference

H1. Local settings file model

Section	Fields observed	Consumers
BusinessProfile	RestaurantName, Phone, Address, WebsiteDomain, SocialMedia, LogoPath, HomepageBackgroundImagePath, TicketFooterText, TaxIdLegalInfo	Desktop branding, role screen, admin screens, tickets, server portal config and served assets.
CurrencyPricing	DefaultCurrencyDisplayMode, UsdToFcRate, ExchangeRateLastUpdatedUtc, RoundingLine, RoundingSubtotal, RoundingGrandTotal, TaxPercent, ServicePercent	Order totals, tickets, dashboard, server portal config, money display.
NavigationBackgrounds	PageImagePath, DimStrength, ContrastIntensity	Main view and visual presentation layers.
Database	Provider, PostgreSQLConnectionString	Database startup configuration for desktop and indirectly API.

H2. Environment variable behavior

Variable	Purpose	Observed use
ELITE_DB_PROVIDER	Select DB provider	Must equal <code>PostgreSql</code> for env-based connection activation.
ELITE_POSTGRES_CONNECTION	Provide PostgreSQL connection string	Overrides need for local settings value when provider flag is set.
ELITE_IMPORT_SILENT	Suppress SQLite import popup	Used only in import mode.

H3. Launch settings and service exposure

API launch settings define an HTTP profile on `http://localhost:5223` and an HTTPS profile on `https://localhost:7194;http://localhost:5223`. These are development-oriented defaults, not direct LAN exposure guarantees. External device access depends on actual process binding and firewall configuration.

I. Workflow Deep Dive

I1. Server order creation flow

1. Server logs in with staff id plus PIN using portal type `Server`.
2. Portal loads branding config, assigned tables, products, ready orders, and drafts.

3. Server selects a table, order source, payment currency, discount, and products.
4. Portal optionally checks whether an open check already exists on the table.
5. API validates server ownership of the table and product existence.
6. API either appends to the open check or creates a new `OrderRecord`.
7. Payment totals are recalculated server-side using shared helpers.
8. Table status is reconciled and remains occupied while the check is open.

I2. Draft round-trip flow

1. The portal captures order form state and the cart as a JSON snapshot.
2. A server draft is saved with a generated human-readable label.
3. The database stores the snapshot as `PayloadJson` in `SharedOrderDrafts`.
4. Loading a draft restores the table, notes, currency, discount, append mode, and cart.
5. The UI explicitly resets search and category filters so the full menu remains visible after load.

I3. Payroll computation flow

1. Determine scheduled workdays and hours for the month from the employee shift pattern.
2. Count absences and late days from attendance rows.
3. Convert every four late days into one deduction unit.
4. Compute base salary after attendance deductions.
5. Compute a five percent bonus on the employee's generated sales where relevant.
6. Subtract applicable salary advances.
7. Write a payroll payment snapshot and, when positive, a salary expense transaction.

I4. Financial posting flow

Completed orders can create revenue entries in `Transactions`. The code uses a structured justification string to avoid duplicate revenue postings. A helper also aligns revenue transaction dates with order completion timestamps when necessary. This indicates that financial records are expected to reflect the business date of actual order completion rather than simple creation time.

J. Tooling and Maintenance Commands

Tool	Typical purpose	Observed caution
<code>dotnet run --project EliteRestaurant.Api</code>	Run API, Swagger, and browser portal host.	May need explicit URL binding for LAN testing.
<code>dotnet run --project EliteRestaurantPro</code>	Launch full desktop application.	May prompt for PostgreSQL connection if settings are missing.
<code>dotnet run --project SeedRunner</code>	Reset and seed PostgreSQL data.	Destructive to existing operational data.
<code>dotnet run --project SeedRunner -- --reduce-active-orders=N</code>	Reduce current active order count while preserving newest N active checks.	Mutates production-like order states; should be used intentionally.

K. Technical Observations and Risks

K1. PostgreSQL-first with SQLite residue

The active runtime clearly expects PostgreSQL, but the codebase still contains significant SQLite-era material: import routines, disabled schema methods under conditional compilation, and at least one utility that appears to expect a `DatabasePath`. This is not inherently wrong, but it means new contributors should not assume the code has been fully normalized after migration.

K2. Schema evolution without migrations

Startup-driven schema mutation is workable for small teams, but it has tradeoffs. There is no migration history, rollback strategy, or environment-to-environment migration discipline represented in the repo. For local development and direct operator-managed installations, this may be acceptable. For larger deployments or strict change management, it is a notable limitation.

K3. Web portal maintainability

The portal is efficient in deployment terms but expensive in long-term maintainability. Styling, UI structure, client state, and API calls are all in one file. Changes are easy to ship quickly, but the file will become progressively harder to test and reason about as portal scope grows.

K4. Auth simplicity

The auth model is perfectly understandable but intentionally lightweight. It is suitable for a trusted local environment, especially when the host machine is physically controlled. However, it is not yet a hardened API authentication design. If future scope includes internet exposure, cross-site access, or multiple organizations, auth should be reconsidered early rather than late.

K5. Mixed UI and shared-core responsibility

`EliteRestaurantPro` is both an application and a reusable domain source. That is convenient today, but it means a web-facing API depends on a project that also contains WPF-specific concerns. A future restructuring into separate class libraries for domain, data access, and desktop UI would reduce coupling.

L. Suggested Documentation Maintenance Process

1. Update the main technical document whenever new modules, deployment practices, or security assumptions change.
2. Update this appendix whenever entity fields, settings structure, endpoint shapes, or tool behaviors change.
3. Re-audit `AppDbContext.Initialize()` and controller routes after major feature additions.
4. Keep screenshots, operator guides, and business SOPs separate from this technical set to avoid mixing audiences.

Final technical note: this appendix is based on the code currently present in the repository and emphasizes actual observed behavior. Where legacy or stale paths

appear to remain, they are called out as observations rather than asserted defects unless confirmed by execution.

Generated on 2026-04-15 for the repository at `C:\Users\ernes\Documents\EliteRestaurant` .